

Shortest paths

Contents

1	Homework solutions	1
1.a	Eulerian graphs	1
1.b	Matchings and vertex covers	1
1.c	Cliques and colorings	2
1.d	ILP formulation of coloring	2
2	Shortest path problem	2
2.a	Statement	2
2.b	ILP formulation	2
2.c	Complexity	3
2.d	Modeling with shortest paths	3
3	Dynamic programming	4
3.a	Bellman's principle	4
3.b	Example: the knapsack problem	4
3.c	Bellman-Ford	4
3.d	Topological sorting	5
3.e	Markov Decision Processes	5
4	The case of non-negative weights	5
4.a	Dijkstra's algorithm	6
4.b	A* algorithm	6
5	Exercises	6

1 Homework solutions

1.a Eulerian graphs

Exercise 3.5: prove the criterion for Eulerian graphs.

If G is a Eulerian graph, then it has a Eulerian cycle (containing every edge exactly once). Each vertex v appears in this cycle a certain number $k_v \geq 1$ of times (not 0). This means that $\deg(v) = 2k_v$.

If every vertex has even degree, we can construct a Eulerian cycle. Start from any vertex v_1 and iteratively pick uncrossed edges until you are stuck. You will necessarily be stuck in v_1 again because the even degree of the vertices implies that for every way in there is a way out. If there is a vertex v_2 on the cycle that has uncrossed incident edges, start a new cycle from v_2 and join it with the previous one. This allows us to cross all edges of the connected component.

1.b Matchings and vertex covers

Exercise 3.9: show that $\nu(G) \leq \tau(G)$.

Let $M \subseteq E$ be a maximum matching and $C \subseteq V$ be a minimum vertex cover. For each edge $e \in M$, pick one vertex $v \in C$ such that e is incident to v . Such a vertex exists because C is a vertex cover. Two edges $e' \neq e$ in M are disjoint because M is a matching. Therefore, the mapping $e \in M \mapsto v \in C$ we defined is injective, and $\nu(G) = |M| \leq |C| = \tau(G)$.

1.c Cliques and colorings

Exercise 3.12: show that $\omega(G) \leq \chi(G)$.

Let K be a clique in G with size k . In any feasible coloring, the vertices of the clique must get different colors, so we need at least k colors. Therefore, $\omega(G) \leq \chi(G)$.

1.d ILP formulation of coloring

Exercise: model the coloring problem as an ILP.

Assume that the coloring of $G = (V, E)$ requires at most p colors:

$$\min_{x,y} \sum_{c=1}^p y_c \quad \text{s.t.} \quad \begin{cases} \sum_{c=1}^p x_{v,c} = 1 & \forall v \in V \\ x_{u,c} + x_{v,c} \leq 1 & \forall (u,v) \in E \quad \forall c \in [1,p] \\ \sum_{v \in V} x_{v,c} \leq y_c & \forall c \in [1,p] \\ x_{v,c} \in \{0,1\} & \forall v \in V \quad \forall c \in [1,p] \\ y_c \in \{0,1\} & \forall c \in [1,p] \end{cases} \quad (1)$$

Based on the previous exercise, can we add inequalities?

2 Shortest path problem

2.a Statement

Input: a directed graph $D = (V, A)$, a cost function $c : A \rightarrow \mathbb{R}$, two vertices o and d .

Output: an $o \rightarrow d$ path P of minimum cost $c(P) = \sum_{a \in P} c(a)$ (or a proof that none exists because o and d are not connected)

We denote by $c(v)$ the cost of a shortest $o \rightarrow v$ path.

In cases where the problem is not well-defined (example: all weights are negative), we will ask for the shortest *simple* or *elementary* path instead.

2.b ILP formulation

Let $x_{u,v}$ be a binary variable equal to 1 if edge (u,v) is part of the path we choose, and 0 otherwise. The shortest *simple* path problem can be stated as:

$$\min_x \sum_{(u,v) \in A} c(u,v)x_{u,v} \quad \text{s.t.} \quad x \in \{0,1\}^A \text{ defines an } o \rightarrow d \text{ path} \quad (2)$$

The constraint “ x defines an $o \rightarrow d$ path” can be expressed linearly: if a vertex v different from o and d is visited, then it needs one incoming edge and one outgoing edge:

$$\forall v \in V, \quad \sum_{u \in \mathcal{N}^-(v)} x_{u,v} - \sum_{w \in \mathcal{N}^+(v)} x_{v,w} = \begin{cases} 0 & \text{if } v \neq o, d \\ -1 & \text{if } v = o \\ 1 & \text{if } v = d \end{cases} \quad (3)$$

2.c Complexity

Theorem: The shortest path problem is NP-complete in the general case.

Proof: Reduction from Hamiltonian path to longest simple path, then from longest simple path to shortest simple path. ■

Polynomial cases:

- Unweighted: Breadth-First Search
- Nonnegative costs: Dijkstra's algorithm
- Acyclic:
 - Undirected: forest (at most 1 path between each pair)
 - Directed: topological sorting
- No negative / absorbing cycles:
 - Directed: Bellman-Ford
 - Undirected: T-joints

Algorithm	Naive complexity	Best complexity
Topological sorting	$O(m + n)$	$O(m + n)$
Dijkstra	$O(n^2)$	$O(m + n \log n)$
Bellman-Ford	$O(mn)$	$O(mn)$

2.d Modeling with shortest paths

Various flavors:

- Single source o , multiple destinations: the one studied here
- Single source o , single destination d : almost as hard
- Multiple sources, multiple destinations: harder

Extensions:

- Shortest paths with resource constraints
- Multi-criteria shortest paths
- Shortest paths on transportation networks: timed trips, multiple modes

Exercise 5.12: Consider a finite collection C of intervals on the real line, each associated with a weight. (1) Show that there is a polynomial algorithm to find the subset of mutually disjoint intervals of C which has maximum weight. (2) How does this help you manage an AirBNB rental?

3 Dynamic programming

3.a Bellman's principle

A subtrajectory of an optimal trajectory is itself optimal.

Generalize the problem to derive a recursion called the Bellman equation. Usually done by changing the bounds or adding parameters.

First compute the value of a solution. It is often easy to go back to the minimizer by working your way backwards.

3.b Example: the knapsack problem

You have n items, each with an integer weight w_i and a value v_i . You want to select a subset of them to put in your knapsack of integer capacity c , to maximize the value inside.

This can be formulated as an ILP:

$$\max_x \sum_{i=1}^n v_i x_i \quad \text{s.t.} \quad \sum_{i=1}^n w_i x_i \leq c \quad (4)$$

To apply dynamic programming, we define the “value function” $V(c, n)$ as the maximum value of a knapsack considering the first n items with capacity c . It satisfies the recursion:

$$V(c, n) = \max\{V(c, n-1), V(c - w_n, n-1) + v_n\} \quad (5)$$

We can compute it by filling a 2-dimensional table in (c, n) .

Exercise: How do we recover an optimal subset of items?

3.c Bellman-Ford

The Bellman-Ford algorithm is dynamic programming applied to directed graphs without absorbing cycles.

Proposition: Let P be an $o \rightarrow v$ path with k arcs, ending with arc (u, v) , and Q be the subpath of P where the last vertex has been removed. If P is a shortest $o \rightarrow v$ path among those with k arcs, then Q is a shortest $o \rightarrow u$ path among those with $k-1$ arcs.

Proof: Suppose there is another $o \rightarrow u$ path Q' with $k-1$ arcs such that $c(Q') < c(Q)$. Then $P' = Q' \cup (u, v)$ is an $o \rightarrow v$ path with k arcs. Furthermore, P' has a cost $c(P') = c(Q') + c(u, v) < c(Q) + c(u, v) = c(P)$. ■

The length of a shortest $o \rightarrow v$ path satisfies the **Bellman equation**

$$\ell(v, k) = \min\{V(u, k-1) + c(u, v) : u \in \delta^-(v)\} \quad (6)$$

with initial conditions

$$\ell(v, 0) = \begin{cases} 0 & \text{if } v = o \\ -\infty & \text{otherwise} \end{cases} \quad (7)$$

We can compute its values starting from $k = 0$. But when do we stop? Since D has no negative cycles, there is a simple shortest path of length at most $n - 1$. We compute $\ell(v, k)$ for all $v \in V$ and $k \in [0, n]$, and then pick k minimizing $\ell(d, k)$.

By remembering, for each v , the in-neighbor u that achieved the minimum, we can build a shortest-path tree.

3.d Topological sorting

A simpler dynamic programming works for Directed Acyclic Graphs (DAGs).

Proposition: Let D be a DAG, P be an $o \rightarrow v$ path ending with edge (u, v) , and Q be the subpath of P where the last vertex has been removed. If P is a shortest $o \rightarrow v$ path, then Q is a shortest $o \rightarrow u$ path.

Proof: Suppose there is another $o \rightarrow u$ path Q' such that $c(Q') < c(Q)$. Then $P' = Q' \cup (u, v)$ is an $o \rightarrow v$ path. Furthermore, P' has a cost $c(P') = c(Q') + c(u, v) < c(Q) + c(u, v) = c(P)$. ■

The length of a shortest $o \rightarrow v$ path satisfies the **Bellman equation**

$$\ell(v) = \min\{\ell(u) + c(u, v) : u \in \delta^-(v)\} \quad \text{and} \quad \ell(o) = 0 \quad (8)$$

Problem: in which order do we enumerate the vertices? The constraint is that we must compute $\ell(u)$ before $\ell(v)$ if there is an edge (u, v) in A .

Proposition: A digraph $D = (V, A)$ is acyclic iff there exists a total order $\preceq$ (i.e. a numbering of the vertices) such that $(u, v) \in A \Rightarrow u \preceq v$.

Proof: Apply Depth-First Search from all vertices without parents. ■

3.e Markov Decision Processes

See polycopié, or on the board if we have time.

4 The case of non-negative weights

What is the shortest way to travel from Rotterdam to Groningen, in general: from given city to given city. It is the algorithm for the shortest path, which I designed in about twenty minutes. One morning I was shopping in Amsterdam with my young fiancée, and tired, we sat down on the café terrace to drink a cup of coffee and I was just thinking about whether I could do this, and I then designed the algorithm for the shortest path. As I said, it was a twenty-minute invention. In fact, it was published in '59, three years later. The publication is still readable, it is, in fact, quite nice. One of the reasons that it is so nice was that I designed it without pencil and paper. I learned later that one of the advantages of designing without pencil and paper is that you are almost forced to avoid all avoidable

complexities. Eventually, that algorithm became to my great amazement, one of the cornerstones of my fame.

— Edsger Dijkstra

4.a Dijkstra's algorithm

Input: a digraph $D = (V, A)$ and costs $c \in \mathbb{Q}_+^A$.

1. Set $U = \emptyset$ (set of visited vertices)
2. Set $\lambda(v) = 0$ if $v = o$ and $\lambda(v) = +\infty$ otherwise (initialize labels)
3. While $V \setminus U \neq \emptyset$:
 1. Choose $v \in V \setminus U$ such that $\lambda(v) = \min_{v' \in V \setminus U} \lambda(v')$ (choose the closest unvisited vertex according to the label)
 2. Add v to U (visit it)
 3. Set $\lambda(w) = \min\{\lambda(w), \lambda(v) + c(v, w)\}$ for all $w \in \delta^+(v)$ (update neighbor labels)

Output: the vector λ which contains all distances $o \rightarrow v$

Proposition: λ represents a tentative distance estimate:

- For all $u \in U$, $\lambda(u) = \ell(u)$
- For all $w \in V \setminus U$, $\lambda(w) = \min_{u \in U} \ell(u) + c(u, w) \geq c(w)$

Proof: Make a drawing!

Since they hold after initialization, we must only check that these properties are preserved by the loop. Let v be the closest vertex according to the tentative distance, i.e. the one achieving $\min_{v' \in V \setminus U} \lambda(v')$. By the second property, $\lambda(v) = \min_{u \in U} \ell(u) + c(u, v')$, so let u be the minimizer there. Consider any other path from o to v . Let v' be its first vertex outside of U , and u' the one before that. The path $o \rightsquigarrow u \rightsquigarrow v$ has cost $\ell(u) + c(u, v) = \lambda(v)$. The path $o \rightsquigarrow u' \rightsquigarrow v'$ has cost $\ell(u') + c(u', v') = \lambda(v') \geq \lambda(v)$. The remaining path $v' \rightsquigarrow v$ has non-negative cost because $c \geq 0$. Hence $o \rightsquigarrow v' \rightsquigarrow v$ is not strictly better. Therefore, $\lambda(v) = \ell(v)$ and the first property is preserved. The second property is easier to verify. ■

4.b A* algorithm

Idea: speed up Dijkstra using a heuristic $h(v)$ to lower-bound the remaining distance $v \rightarrow d$.

Procedure: grow a set of paths and trim the ones that are hopeless.

Special case of Branch & Bound and LP duality (see later classes).

Essential in transportation networks because the graphs are large but we have a geographical idea of where to go.

5 Exercises

Exercise 5.19: Consider two words w_1 and w_2 (sequences of symbols) on the same alphabet. A subword is an increasing subsequence (not necessarily contiguous) of a word.

Give a polynomial algorithm to find the longest common subword of w_1 and w_2 , and describe its complexity as a function of the lengths n_1 and n_2 .

Exercise 5.20: Suggest a dynamic programming algorithm for the Traveling Salesperson Problem, with time complexity $O(n^2 2^n)$.